Recursion

17 July 2025 20:37

What is Recursion in Java?

Recursion is a programming technique where a method (function) calls itself in order to solve a problem. Instead of using loops (for, while), We can sometimes solve problems using recursions, especially when the problem can be broken down into smaller sub problems of the same kind.

Recursive methods has two parts:
1. Base Case(Stopping Condition): A condition that stops the recursion to prevent infinite calls.
2. Recursive Case: The part where the method calls itself with a smaller or simpler input.

Think of like this: Solving a big problem by breaking it into smaller versions of the same problem until you reach the simplest form(base case).

Simple example: Recursive function to print numbers from n to one.

```
package Recursion;
/**
 * Write a description of class PrintNumbersInDescending here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class PrintNumbersInDescending
{
    public static void printDescending(int n){
        if(n == 1){ //base case or exit case
            System.out.print(n);
            return;
        }
        System.out.print(n+", ");
        printDescending(n-1);  //recursive call
    }

    public static void main(String args[]){
        System.out.print("\fEnter any positive integer: ");
        printDescending(new java.util.Scanner(System.in).nextInt());
    }
}
```

n = 10
printDescending(9)
printDescending(8)
printDescending(1)

Note: recursion always returns to previous, when return statement is encountered.

Output:

BlueJ: Terminal Window - AaditClass7

Options

```
Enter any positive integer: 10
10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

Another example: Factorial using Recursion

n! = n * (n-1) * (n-2) * … * 1

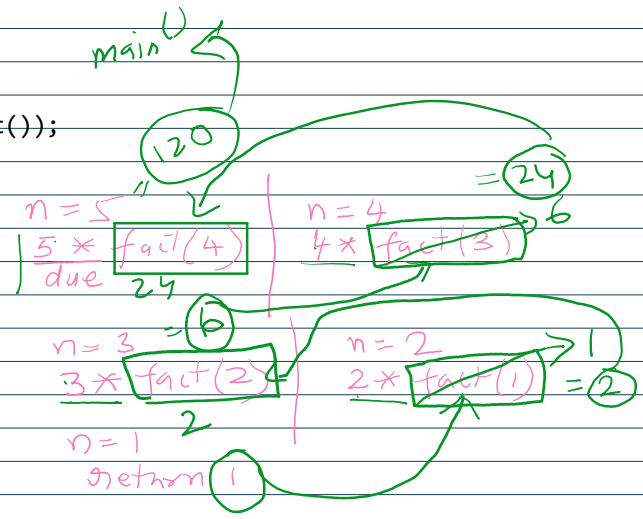
Example: fact(5) = 5 * 4 * 3 * 2 * 1 = 120

```
package Recursion;


/**
 * Write a description of class Factorial here.
 *
 * @author (your name)
```

```
 * @version (a version number or a date)
 */
public class Factorial
{
    public static void main(String args[]){
        System.out.print("\fEnter any integer value: ");
        int f = fact(new java.util.Scanner(System.in).nextInt());
        System.out.print("\nFactorial = " + f);
    }

    public static int fact(int n){
        if(n == 1 || n == 0){ //base case or exit case
            return 1;
        }
        else{
            return n * fact(n-1); //recursive call
        }
    }
}
```



For that, recursive methods must have a base case or they will lead to infinite recursion and eventually a stack overflow error.
Recursion can be elegant, but may use more memory than loops.

When to use recursion?

- **Problems with repetitive patterns (Example factorial, Fibonacci).**
- **Divide and conquer problems. (example searching, sorting, etc.)**
- **Tree or graph traversals.**

Q1. Print the numbers from 1 to N using Recursion(Ascending order)
Q2. WAP to find the Sum of First N Natural Numbers. For example:
Input: 5
Output: Sum = 15 [5 + 4 + 3 + 2 + 1]

Home work:

Recursion Practice Questions in Java

**1. Sum of Digits (Recursive Function)**

Write a recursive Java method to calculate the sum of the digits of a given number.
****Example:****
Input: `1234` → Output: `1 + 2 + 3 + 4 = 10`

**2. Count Occurrence of a Character in a String**

Write a recursive function that counts how many times a given character appears in a string.
****Example:****
Input: `"bananas"`, character = `'a'` → Output: `3`

**3. Find the GCD of Two Numbers**

Write a recursive method to calculate the **Greatest Common Divisor (GCD)** of two integers using **Euclid's Algorithm**.
Example:
GCD of `48` and `18` = `6`

---

4. Print All Elements of an Array in Reverse Order

Create a recursive function to print the elements of an array in **reverse order**.
Example:
Input: `[1, 2, 3, 4]` → Output: `4 3 2 1`

---

5. Recursively Check if a String is a Palindrome

Create a recursive method to check whether a string is a **palindrome** (reads the same forwards and backward).
Example:
Input: `"racecar"` → Output: `true`

---

6. Generate All Subsets of a String

Write a recursive function that prints all **possible subsets** (also called powersets) of a given string.
Example:
Input: `"ab"` → Output: `["", "a", "b", "ab"]`

---

7. Calculate the Nth Term of a Fibonacci Series

Write a recursive function that calculates the `n`th term in a Fibonacci series using recursion.
Example:
Input: `n = 6` → Output: `5`
Fibonacci: `0 1 1 2 3 5`

---

8. Recursive Power Function (x^n)

Write a recursive function to compute `x` raised to the power `n` (i.e., `x^n`) without using loops or built-in methods.
Example:
`power(2, 4)` → Output: `16`

---

9. Print All Numbers from 1 to N Without Loop

Use recursion to print all numbers from `1` to `N`.
Do not use any loops.
Example:
Input: `n = 5` → Output: `1 2 3 4 5`

---

10. Recursively Remove All Duplicate Characters from a String

Write a recursive Java method to remove all duplicate characters from a string while maintaining the original order.
Example:

Input: `"abacabad"` → Output: `"abcd"`